LICENCE BUSINESS INTELLIGENCE

Python, ETL et entrepôts de données

Atlas Papeterie : un seul projet, des premiers gestes au rapport de ventes.

28 h

70 % pratique · 30 % théorie

Youssef Douma · Leaders University

Vous allez construire quelque chose d’utile

1

Lire

Ouvrir des fichiers, repérer les colonnes et expliquer ce qu’une ligne représente.

2

Fiabiliser

Nettoyer, calculer, contrôler et charger un entrepôt de données.

3

Expliquer

Produire un rapport et formuler une décision avec ses limites.

70 % de pratique = 19 h 36 · 30 % de théorie = 8 h 24.

Atlas Papeterie prépare ses prochains achats

1

Deux boutiques

NBE = Nabeul
TUN = Tunis
Des ventes sur plusieurs jours.

2

Quatre produits

Carnet · Stylo
Classeur · Surligneur
Des prix en dinars tunisiens.

3

Une question

Quels produits suivre avant de décider des prochains achats ?

Atlas Papeterie et les données sont fictifs. Aucune donnée personnelle.

Du fichier brut au rapport lisible

1

Fichiers CSV

Les ventes arrivent avec quelques erreurs volontaires.

2

Entrepôt SQLite

Les tables sont reliées et les montants contrôlés.

3

Rapport HTML

Un navigateur affiche les résultats et leur contexte.

Tout fonctionne sur votre ordinateur après le téléchargement initial.

Prédire → essayer → expliquer

1

Avant

Je prédis l’effet d’un petit changement et je l’écris.

2

Pendant

Je change une seule chose, j’enregistre et j’exécute.

3

Après

Je compare, j’explique, puis je restaure le cas de référence.

Le professeur vous aide à comprendre. Aller plus vite n’est pas l’objectif.

Un dossier, un fichier, un programme

1

Dossier

Il contient d’autres éléments.
Exemple : scripts/

2

Fichier

Il porte un nom et une extension.
Exemple : 01_premiers_pas.py

3

Programme

Python lit les instructions du fichier .py dans leur ordre.

Extraire tout le ZIP avant de travailler. Ne pas exécuter depuis le ZIP.

Où écrire ? Où lire le résultat ?

ÉDITEUR · EN HAUT

Je modifie mon fichier .py.

Ctrl+S : enregistrer.
F5 : exécuter le fichier.

SHELL · EN BAS

Je lis les résultats.

Le symbole >>> indique une console Python.

Ouvrir scripts/01_premiers_pas.py avec File → Open dans Thonny.

Deux carnets à 4.500 TND : 9.000 TND

```
quantite = 2
prix_millimes = 4500
montant = quantite * prix_millimes
print(montant)
# 9000 millimes = 9.000 TND
```

1. Une variable donne un nom à une valeur.
2. L’étoile * signifie multiplier.
3. print affiche une valeur dans le Shell.

1 TND = 1 000 millimes. Nous stockerons l’argent en nombres entiers.

Changez une quantité, pas tout le script

01

Si l’on vend 3 carnets au lieu de 2, quel montant attendez-vous ?

1. Dans 01_premiers_pas.py, repérer la quantité et prédire le résultat.
2. Modifier la valeur, enregistrer, puis exécuter avec F5.
3. Expliquer la différence, puis rétablir la valeur de départ.

À ce stade, je sais ouvrir, modifier, enregistrer et exécuter un fichier.

Une table se lit ligne par ligne

sale_id	date	shop_id	product_id	quantité
S001	2026-01-05	NBE	P01	2
S002	2026-01-05	NBE	P02	3
S003	2026-01-05	TUN	P03	1

Une colonne donne un type d’information. Une ligne rassemble une vente.

Que représente exactement une ligne ?

1

Une vente

Un identifiant sale_id, un produit, une boutique et une date.

2

Une quantité

2 carnets peuvent appartenir à une seule ligne de vente.

3

Un montant

Quantité × prix unitaire. Ce montant est une vente, pas un bénéfice.

12 lignes de vente ne signifient ni 12 clients ni 12 commandes.

Python peut parcourir un fichier CSV

```
import csv

with open("exemple.csv",
          encoding="utf-8-sig") as f:
    lecteur = csv.DictReader(f)
    lignes = list(lecteur)
print(len(lignes))
```

1. CSV : du texte organisé en colonnes.
2. DictReader utilise la première ligne comme noms de colonnes.
3. len compte les lignes lues.

Illustration : exemple.csv doit exister. Utiliser 02_lire_csv.py pour le vrai fichier.

Lire avant de calculer

02

Le fichier brut contient 16 lignes. Sont-elles toutes exploitables ?

1. Ouvrir puis exécuter scripts/02_lire_csv.py.
2. Repérer sale_id, quantity et unit_price_tnd dans les premières lignes.
3. Décrire une ligne à votre voisin, sans simplement réciter ses valeurs.

À ce stade, je sais distinguer les noms de colonnes des valeurs.

Même boutique, écritures différentes

VALEURS BRUTES	VALEUR HARMONISÉE
" nbe "	NBE
"NBE"	NBE
"nbe"	NBE

strip enlève les espaces autour. upper transforme les lettres en majuscules.

Une erreur doit avoir une raison visible

1

Quantité nulle

I001 : quantity = 0
La règle exige un entier strictement positif.

2

Produit inconnu

I002 : product_id = P99
Ce code n’existe pas dans le catalogue.

3

Date impossible

I003 : 2026-02-30
Cette date n’existe pas.

Rejeter une ligne n’est pas l’effacer : conserver la ligne et sa raison.

Chaque ligne d’entrée reçoit une destination

16
lignes brutes

12
retenues

1
doublon exact

3
rejets expliqués

16 = 12 + 1 + 3

Le doublon de S003 ne correspond pas à une nouvelle vente.

Contrôler un résultat, puis l’expliquer

03 Pourquoi ne suffit-il pas d’annoncer « il reste 12 lignes » ?

1. Exécuter 03_nettoyer.py et lire les fichiers de contrôle dans outputs/03/.
2. Retrouver une ligne rejetée et expliquer sa règle en français.
3. Vérifier l’égalité : retenues + doublons + rejets = lignes brutes.

À ce stade, je sais justifier ce qui a été retenu et ce qui a été isolé.

Le texte « 2 » n’est pas le nombre 2

APRÈS LECTURE DU CSV

quantity = "2"

C’est du texte, même si le contenu ressemble à un nombre.

APRÈS CONVERSION

quantity = 2

C’est un entier : on peut multiplier et additionner.

Convertir après contrôle. Un texte vide ne devient pas une quantité valide.

Des dinars lisibles, des millimes exacts

```
from decimal import Decimal

prix_tnd = Decimal("4.500")
prix_millimes = int(prix_tnd * 1000)
montant_millimes = 2 * prix_millimes
print(montant_millimes)  # 9000
```

1. Decimal lit le prix décimal sans passer par float.
2. La base conserve 4500 millimes par carnet.
3. L’affichage final revient aux TND avec 3 décimales.

Une conversion ne remplace pas la validation du prix et de sa précision.

Une boucle traite plusieurs valeurs

```
quantites = [2, 3, 1]
total = 0
for quantite in quantites:
    total = total + quantite
print(total)  # 6
```

1. Une liste rassemble plusieurs valeurs.
2. for répète les lignes indentées pour chaque valeur.
3. Le total augmente à chaque passage.

Les espaces au début d’une ligne font partie de la structure de Python.

Vérifier les unités avant d’additionner

04

Les 12 ventes retenues donnent 99.000 TND pour 32 articles.

1. Exécuter 04_transformer.py et ouvrir outputs/04/ventes_transformees.csv.
2. Vérifier à la main le montant de S001 : 2 × 4500 millimes.
3. Expliquer pourquoi 32 articles et 12 ventes sont deux mesures différentes.

À ce stade, je sais calculer un montant et nommer son unité.

SELECT choisit ce que l’on veut voir

```
SELECT sale_id, quantity
FROM ventes;
```

1. SELECT : quelles colonnes ?
2. FROM : quelle table ?
3. Une requête lit les données ; elle ne modifie pas ici la table.

SQL sera exécuté par le script 05_sql.py ; ce n’est pas du code Python isolé.

WHERE garde les lignes qui conviennent

```
SELECT sale_id, quantity
FROM ventes
WHERE quantity >= 3;
```

1. La condition est testée pour chaque ligne.
2. Une quantité de 3 est incluse par >= 3.
3. Le fichier source ne change pas.

Prédire : une vente de 2 articles apparaîtra-t-elle dans le résultat ?

Regrouper, additionner, puis lire le sens

1

Regrouper

GROUP BY rassemble les lignes de même boutique.

2

Additionner

SUM additionne les montants de chaque groupe.

3

Interpréter

NBE : 43.500 TND
TUN : 55.500 TND
Sur les mêmes 3 jours.

43.500 + 55.500 = 99.000 TND. Le total doit être conservé.

Poser une question précise aux données

05

Que change le passage de la boutique NBE à la boutique TUN ?

1. Ouvrir 05_sql.py et repérer BOUTIQUE = "NBE".
2. Prédire le résultat, remplacer NBE par TUN, puis exécuter.
3. Comparer 43.500 TND et 55.500 TND, puis rétablir NBE.

À ce stade, je sais lire et adapter une petite requête.

Pourquoi séparer faits et dimensions ?

1

Les faits

Ce qui a été mesuré : une quantité et un montant pour une vente.

2

Les dimensions

Le contexte : quel produit, quelle boutique et quelle date ?

3

Les liens

Des clés relient chaque fait aux lignes qui décrivent son contexte.

Notre entrepôt pédagogique est un fichier SQLite : outputs/atlas.sqlite.

Définir la ligne avant de créer la table

Question	Réponse pour Atlas
Une ligne de faits ?	Une vente élémentaire identifiée par sale_id
Un produit ?	Un seul produit par vente élémentaire
Une quantité ?	Plusieurs articles possibles sur cette ligne
Une période ?	Une date de vente précise

Si le sens d’une ligne change, les additions peuvent devenir trompeuses.

Trois dimensions autour d’une table de faits



1 → N : une dimension décrit plusieurs ventes.

Une vente possède une seule référence de produit, de boutique et de date.

Code métier et clé substitut : deux rôles

Catalogue métier	Dimension	Fait de vente
P01 = Carnet	product_key = 1	S001 → product_key 1
P02 = Stylo	product_key = 2	S002 → product_key 2
Code du produit	Clé locale de liaison	Référence la dimension

P01 identifie le produit métier ; product_key sert aux relations de notre entrepôt.

Dessiner avant de charger

06

Où placer le nom « Carnet » et où placer la quantité 2 ?

1. Dessiner les quatre tables avec trois liens dimension → faits.
2. Placer product_name dans dim_product et quantity dans fact_sales.
3. Exécuter 06_modele.py et comparer le modèle au dessin.

À ce stade, je sais distinguer une mesure de son contexte.

Les dimensions d’abord, les faits ensuite

1

1 · Dimensions

Créer les produits, les boutiques et les dates utiles.

2

2 · Clés

Retrouver la clé interne qui correspond à chaque code métier.

3

3 · Faits

Insérer la vente avec ses références et ses mesures.

Une clé étrangère ne doit pas pointer vers une dimension absente.

S001 garde son sens pendant le chargement

Information source	Dans l’entrepôt	Signification
S001	sale_id = S001	La même vente
P01	product_key → Carnet	Le produit décrit
NBE	shop_key → Nabeul	La boutique décrite
2 × 4.500 TND	9000 millimes	Le même montant

Les clés internes peuvent changer d’un modèle à l’autre ; le sens doit rester stable.

Trois preuves valent mieux qu’un « terminé »

1

Volume

12 faits chargés
32 articles
Aucune vente valide perdue.

2

Montant

99000 millimes
= 99.000 TND
Le total reste le même.

3

Relations

Chaque fait retrouve sa date, son produit et sa boutique.

Une transaction protège la cohérence : en cas d’erreur, on annule le lot.

Une jointure retrouve le nom du produit

DANS LE FAIT

S001
product_key = 1
amount_millimes = 9000



APRÈS LA JOINTURE

S001
Carnet
9.000 TND

JOIN associe les lignes dont la clé correspond. Il ne crée pas une nouvelle vente.

Relier les faits et les noms, puis additionner

```
SELECT p.product_name,
       SUM(f.amount_millimes)
FROM fact_sales AS f
JOIN dim_product AS p
  ON f.product_key = p.product_key
GROUP BY p.product_name;
```

1. JOIN : joindre la dimension produit.
2. ON : préciser les deux clés qui correspondent.
3. GROUP BY : totaliser par nom de produit.

Requête illustrative dans SQLite : le résultat de SUM est encore en millimes.

Montrer que l’entrepôt respecte les ventes

07

Le fichier SQLite existe. Comment prouver qu’il contient les bonnes données ?

1. Exécuter 07_charger.py et relever le nombre de faits et le montant.
2. Vérifier : 12 faits, 32 articles, 99.000 TND.
3. Rejouer le chargement et expliquer pourquoi le total ne doit pas doubler.

À ce stade, je sais contrôler un chargement au lieu de seulement le lancer.

Un nouveau fichier contient aussi une ancienne vente

sale_id	Contenu	Action attendue
S013	1 carnet · 4.500 TND	Ajouter la nouvelle vente
S014	2 classeurs · 12.000 TND	Ajouter la nouvelle vente
S012	Vente déjà chargée · 18.000 TND	Reconnaître la même vente

Un identifiant stable aide à éviter de compter deux fois la même vente.

Le deuxième passage doit conserver le résultat

Avant ajout

12 faits

99.000 TND

Premier passage

14 faits

115.500 TND

Deuxième passage

14 faits

115.500 TND

Idempotent : refaire la même opération n’ajoute pas une deuxième fois les mêmes faits.

Même identifiant, contenu différent : arrêter

DÉJÀ CHARGÉ

S012
3 classeurs
18.000 TND

→

NOUVEL ARRIVAGE MODIFIÉ

S012
4 classeurs
24.000 TND

Ce n’est ni une nouvelle vente ni un doublon exact : le lot doit signaler le conflit.

Tester le premier passage et le rejeu

08

Pourquoi 3 lignes entrantes donnent-elles seulement 2 ventes nouvelles ?

1. Exécuter 08_incremental.py et noter les ajouts et lignes déjà présentes.
2. Rejouer le script : attendre 0 nouvel ajout et 3 lignes déjà présentes.
3. Vérifier le total : 14 faits, 35 articles, 115.500 TND.

À ce stade, je sais expliquer un chargement incrémental et son rejeu.

Trois mesures, trois significations

14

ventes élémentaires après l’ajout

35

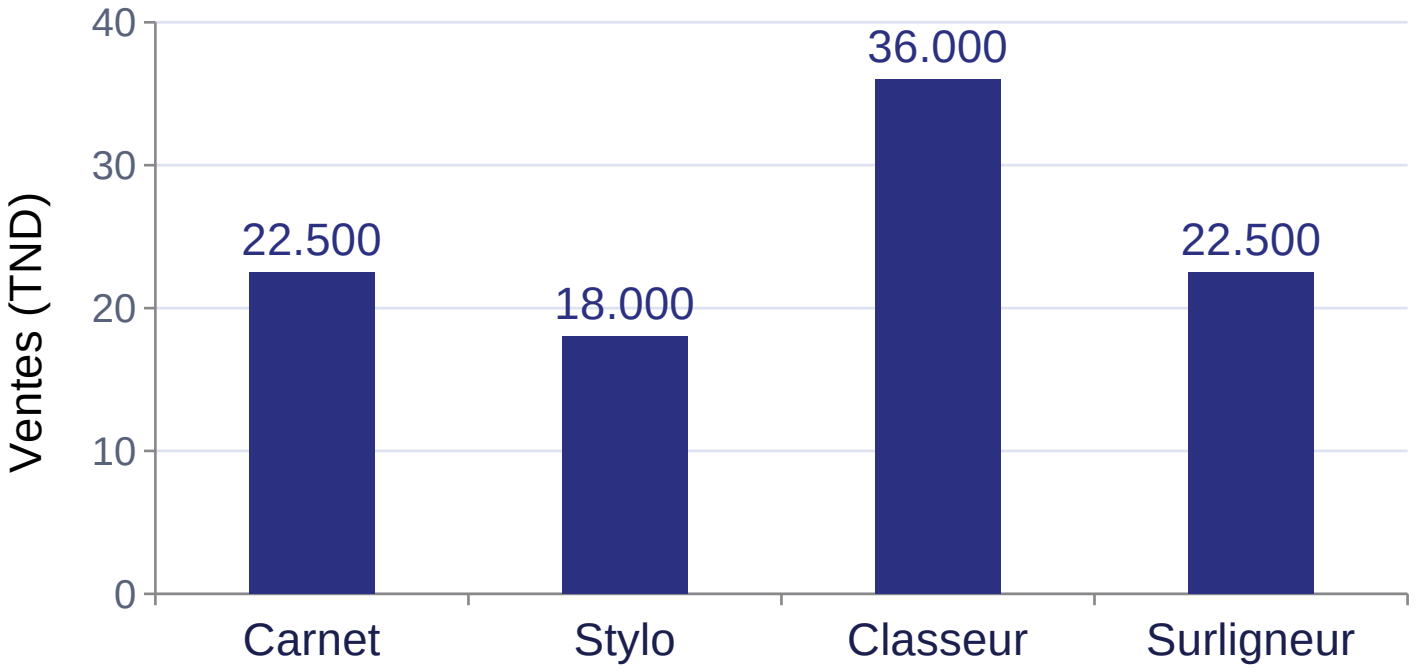
articles vendus après l’ajout

115.500

TND de ventes après l’ajout

Le rapport indique la période et l’unité. Il lit les données de l’entrepôt.

Quel produit réalise le plus de ventes ?



36.000 TND

Le classeur est premier en montant vendu sur ces 3 jours.

Cas initial uniquement · 5–7 janvier 2026 · 12 ventes · Total : 99.000 TND.

Dire ce que le chiffre permet de conclure

1

J’observe

Sur les trois jours initiaux, le classeur est premier en montant vendu.

2

Je propose

Examiner son stock et les délais du fournisseur avant le prochain achat.

3

Je limite

Ces ventes ne suffisent pas à connaître la marge, le stock ou la demande future.

Une recommandation utile distingue une observation, une action et une limite.

Produire puis lire le rapport

09

Une personne qui ne connaît pas Python peut-elle comprendre votre résultat ?

1. Exécuter 09_rapport.py puis ouvrir outputs/rapport_atlas.html.
2. Repérer le nombre de ventes, les articles, le montant et la période.
3. Rédiger trois phrases : observation, proposition, limite.

À ce stade, je sais transformer un calcul en explication compréhensible.

Réparer, rejouer, présenter

1

Réparer

Dans la copie CSV de outputs/10/, corriger S015 : quantité 0 → 2.

2

Rejouer

Choisir la copie réparée dans SOURCE_AJOUT et exécuter 10_defi.py.

3

Présenter

Contrôler 15 faits, 37 articles et 118.500 TND. Expliquer le changement.

Séance finale : 1 h prévue, dont 42 min de pratique et 18 min de synthèse.

Montrer le chemin, pas seulement l’écran final

1

Avant

Le symptôme, la donnée ou l’hypothèse qui posait problème.

2

Changement

La correction réalisée et la raison de ce choix.

3

Après

Le résultat contrôlé, avec une valeur, une unité et une limite.

Une autre solution est possible si vous pouvez la montrer, la vérifier et l’expliquer.

Extraire, transformer, charger : vous l’avez fait

1

Extract

Lire les CSV et conserver les sources intactes.

2

Transform

Harmoniser, contrôler, isoler les rejets et calculer les montants.

3

Load

Alimenter les dimensions puis les faits ; rejouer sans doublonner.

Le rapport vient ensuite : il exploite les données préparées dans l’entrepôt.

Six mots pour expliquer votre projet

Mot	Explication courte
CSV	Fichier texte structuré en colonnes
ETL	Extraire, transformer, charger
Fait	Une mesure au grain défini
Dimension	Le contexte qui décrit les faits
Clé	Une valeur utilisée pour identifier ou relier
Incrémental	Ajouter les nouvelles données sans tout recompter

Expliquer avec les ventes Atlas, puis seulement employer le terme technique.

Lire le message avant de changer le code

1

Fichier introuvable

Le dossier a-t-il été extrait ? Avez-vous ouvert le bon fichier .py ?

2

SyntaxError

Vérifier les guillemets, les parenthèses et la ligne indiquée.

3

IndentationError

Vérifier les espaces au début des lignes dans la boucle ou la condition.

Dire : fichier → action → message exact → essai effectué. Une erreur est une information.

Retrouver votre point de départ

1

Le bon PDF

Lire le guide 01 pour l’installation, puis ouvrir le cours 02.

2

Le bon fichier

Ouvrir le script de la mission, enregistrer et exécuter avec F5.

3

La bonne preuve

Comparer le résultat attendu avant de poursuivre la mission suivante.

Le document 03 aide à dépanner ; le document 04 propose les indices et corrigés.

Vous avez les bases pour une prochaine étape

1

pandas

Manipuler des tables plus grandes avec une bibliothèque spécialisée.

2

Power BI

Importer les résultats pour construire un tableau de bord interactif.

3

Historisation

Conserver les changements de description d’une dimension.

Ces extensions ne sont pas nécessaires pour réussir le projet de 28 h.

DU FICHIER À LA DÉCISION

Vous savez relier les données à une décision.

1. Lire et contrôler des fichiers de ventes.
2. Construire et alimenter un petit entrepôt.
3. Rejouer un pipeline sans compter deux fois.
4. Montrer un résultat et expliquer ses limites.

Votre prochaine étape : refaire le parcours sur une copie de vos propres données.